# Resolución de juegos

- Juegos de suma cero e información perfecta
- Minimax
- Alpha-beta pruning

# Juegos de suma cero

- La ganancia de un jugador es la pérdida del otro.

# Juegos de suma cero

- Ejemplos:
  - Tateti
  - Ajedrez
  - Backgammon

- Contraejemplos:
  - Dilema del prisionero

# Juegos de información perfecta

- Todos los jugadores tienen acceso a información completa sobre el estado del juego.
- No hay azar.

# Juegos de información perfecta

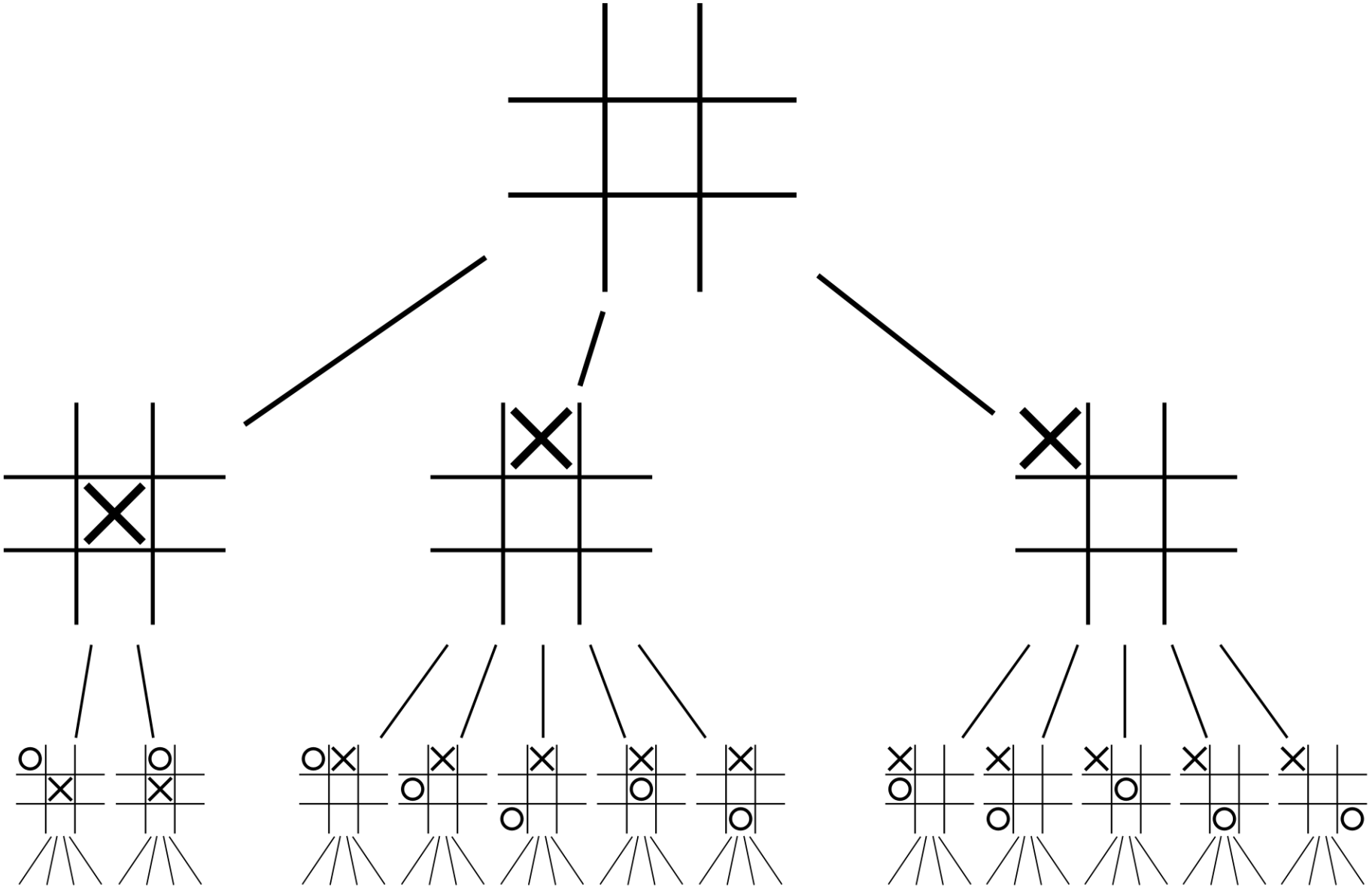
- Ejemplos:
  - Tateti
  - Ajedrez
  - Dilema del prisionero

- Contraejemplos:
  - Backgammon
  - Póker

# Espacio de búsqueda para juegos

- Representamos el juego como un **grafo**.
  - Un **nodo** es un estado del juego.
  - Un **arco** es un movimiento de un jugador.

- Interesa construir un **árbol** para representar el espacio de búsqueda de los **movimientos** posibles a partir de cierto estado.



# Minimax

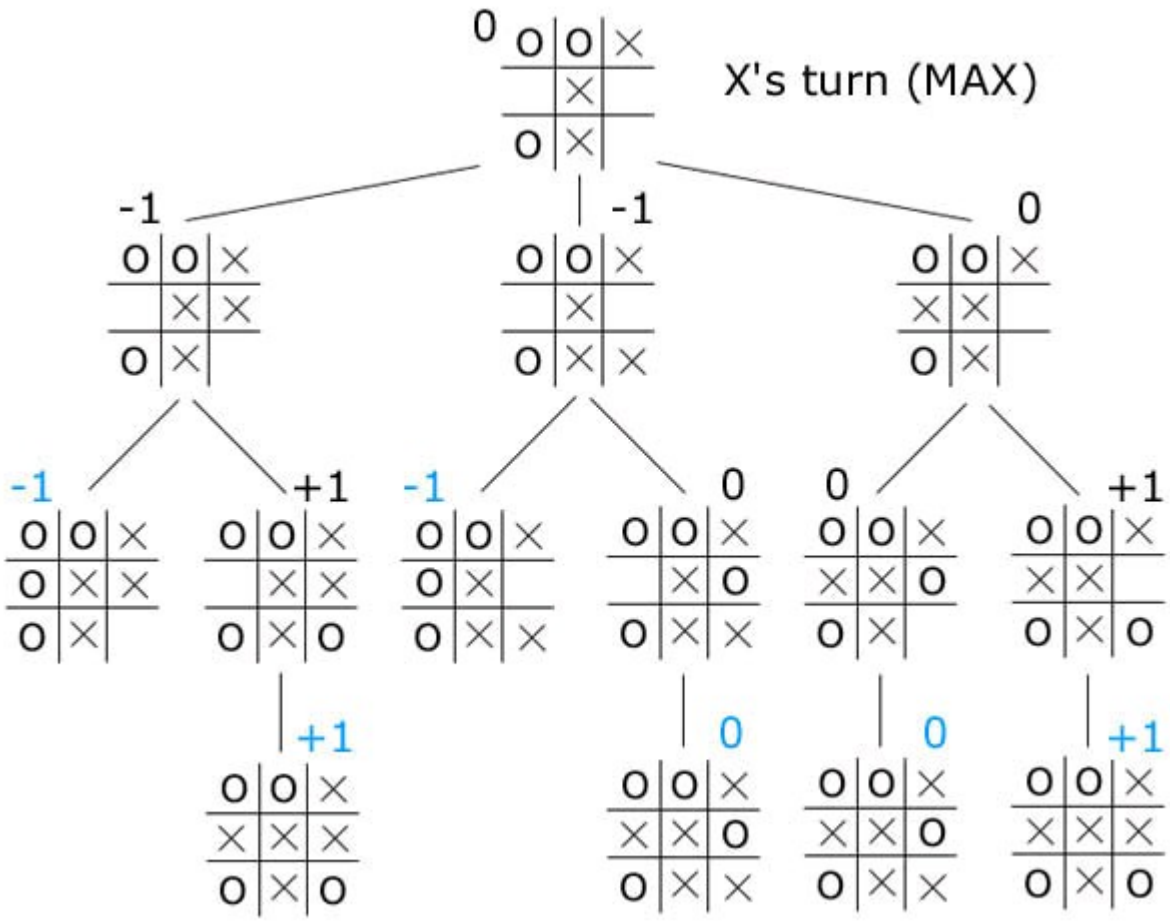
- Método para determinar la mejor jugada posible a realizar por parte de un jugador.
  - Minimizar la pérdida máxima espereda.
- Aplica para juegos de 2 jugadores, por turnos, de suma cero e información perfecta.

# Minimax

- El jugador que debe mover es un nodo **Max**, que quiere maximizar su ganancia.
- La mejor movida para Max es la que maximiza su ganancia, considerando todas las movidas posibles para el otro jugador (Min).
- Suponemos que cada jugador va a hacer el mejor juego posible:
  - Max va a maximizar su ganancia
  - Min va a tratar de que la ganacia de Max sea mínima
- Los nodos del árbol son Max o Min (juega Max o juega Min) alternadamente por niveles.

# Minimax: valor de un nodo

- El valor de cada nodo se determina por una función de evaluación o a partir de los valores de sus hijos
- Para un nodo **MAX**, el valor es el **máximo** de los valores de los hijos
- Para un nodo **MIN**, el valor es el **mínimo** de los valores de los hijos
- ¿Qué pasa si el nodo es una hoja?


0
X's turn (MAX)
-1
-1
0
-1
+1
-1
0
0
+1
+1
0
0
+1

# Minimax: valor de un nodo

- En general, el espacio del juego es demasiado grande como para explorarlo todo
- Se avanza hasta una determinada profundidad y luego se evalúa con una función (heurística)

# Minimax

Para cada jugada de la computadora:

- Nodo **raíz** = nodo Max
- Expansión del árbol hasta cierta profundidad (parámetro)
- Función de evaluación en las hojas (evaluación estática)
- Cálculo por niveles de los valores de los nodos interiores, alternadamente, el valor de un nodo es el máximo o el mínimo de los valores de sus hijos. (evaluación por lookahead)
- El valor elegido para **raíz** determina la movida

# Algoritmo

```
function minimax(node, depth, is_maximizing)

if node is a terminal node or depth = 0
    return the heuristic value of node

else if is_maximizing
    max_value := -∞
    foreach child of node
        max_value := max(max_value, minimax(child,depth-1,false))
    return max_value

else
    min_value := ∞
    foreach child of node
        min_value := min(min_value, minimax(child,depth-1,true))
    return min_value
```

# Minimax

- Una buena función de evaluación es fundamental.
  - En el ajedrez uno de los criterios puede ser la cantidad y valor de piezas
  - En el laboratorio de este año?
- Cuánto tiempo lleva?
  - En el tateti, es posible considerar todas las posiciones
  - Pero en el Ajedrez o en el Go esto no es posible
  - En el laboratorio de este año?

# Minimax con poda del árbol

- Optimización: no expandir algunos subárboles
- ¿Cuándo podemos no expandir?
  - Cuando sabemos que no mejoramos lo que tenemos hasta ahora

# Minimax con poda del árbol

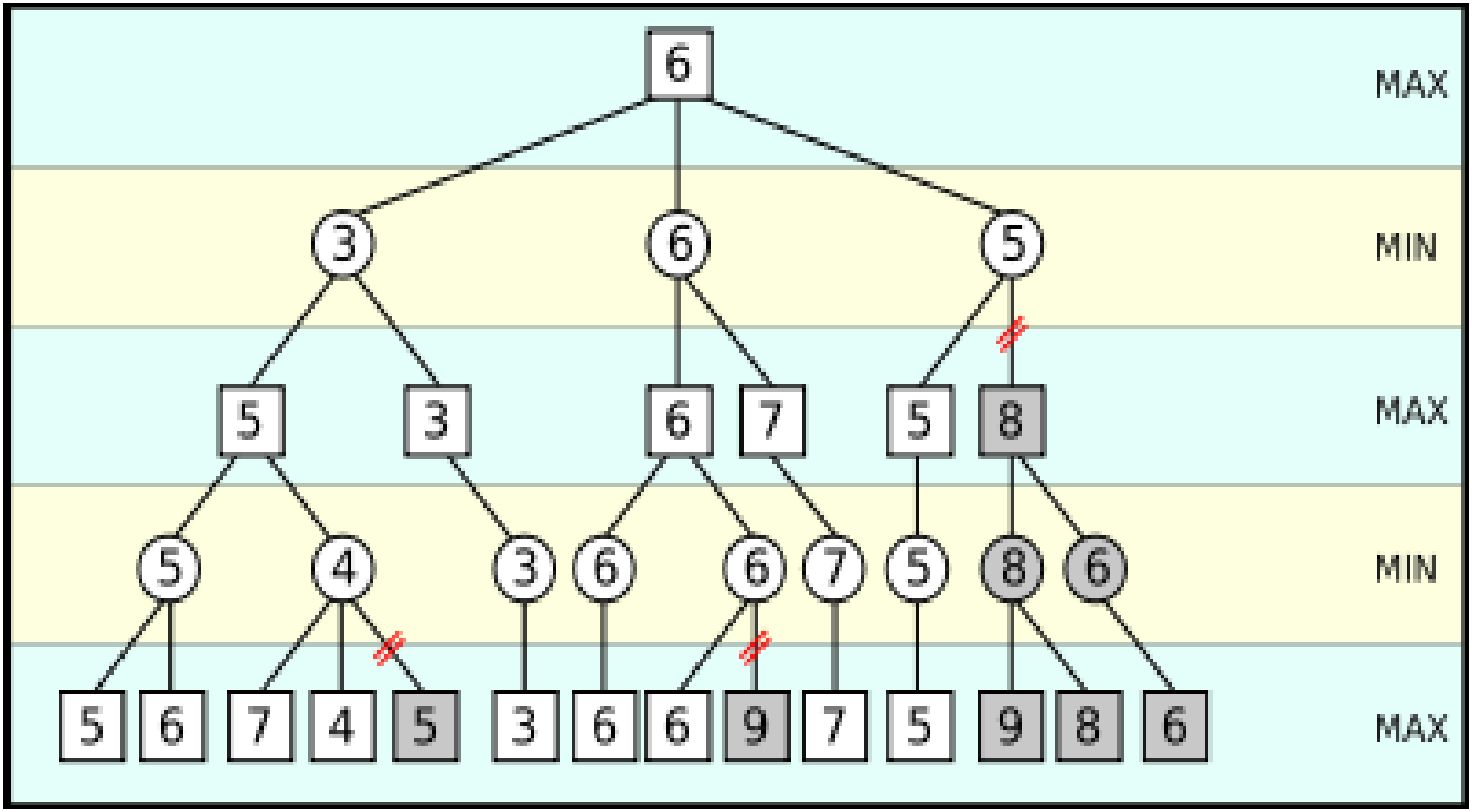
- *alpha:* Valor de mi mejor jugada hasta el momento
- *beta:* Valor de la mejor jugada de mi oponente hasta el momento
- Si *beta* ≤ *alpha*, la mejor movida del oponente obtendría un resultado peor que el que tengo hasta el momento ⇒ podo

# Alpha-beta pruning

**Reglas para  Alpha-beta Pruning**

- **Alpha Pruning:** Descartar la búsqueda debajo de cualquier nodo MIN con beta menor o igual al valor alpha de cualquier de sus ancestros MAX
- **Beta Pruning:** Descartar la búsqueda debajo de  cualquier nodo MAX con alpha mayor o igual al valor beta de cualquiera de sus ancestros MIN

# Alpha-beta pruning




https://en.wikipedia.org/wiki/Alpha-beta_pruning

# Algoritmo

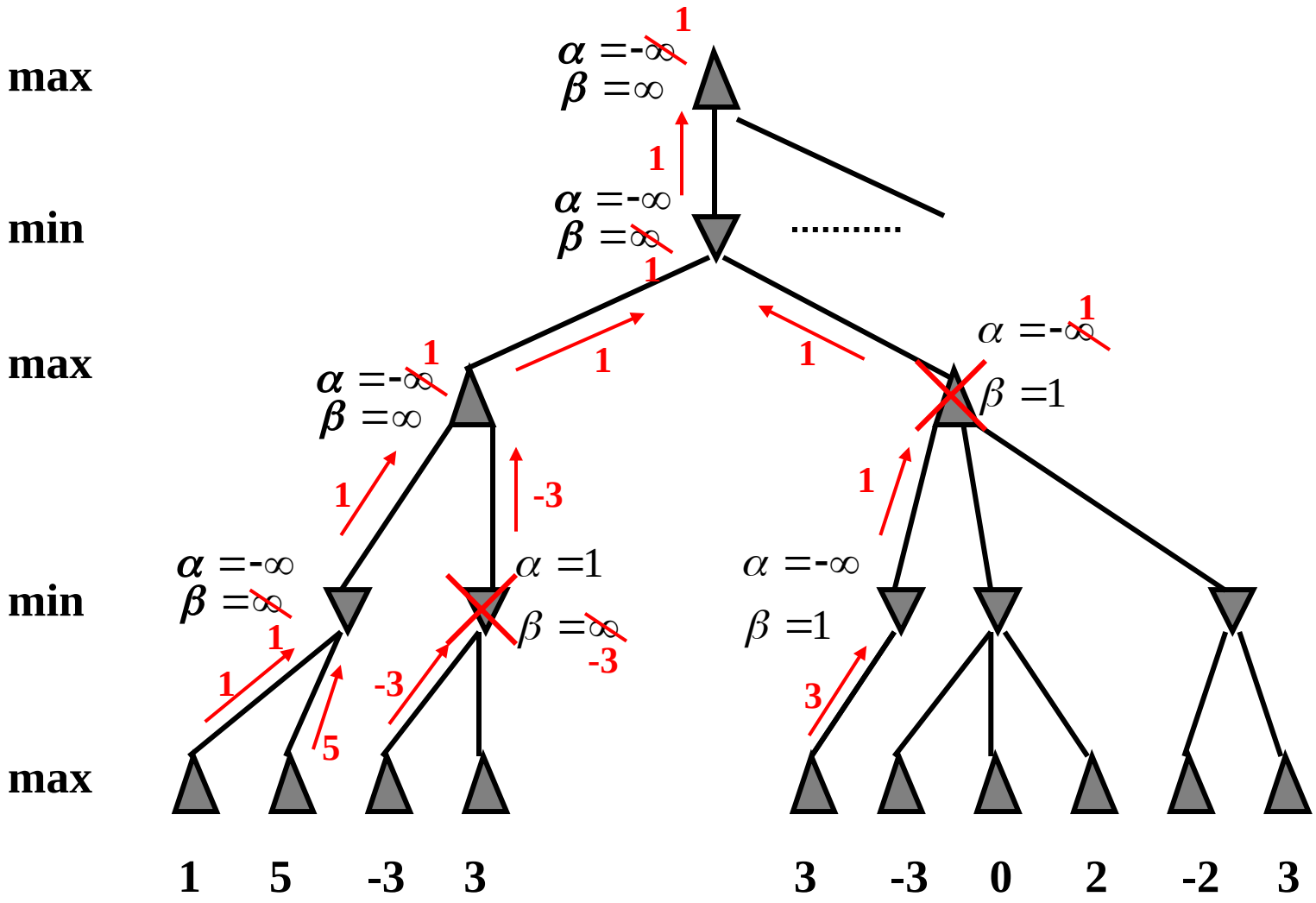
- Inicialmente :
  - alpha es menos infinito
  - beta es infinito

- Cuando la recursión progresa, la ventana se achica

# Algoritmo

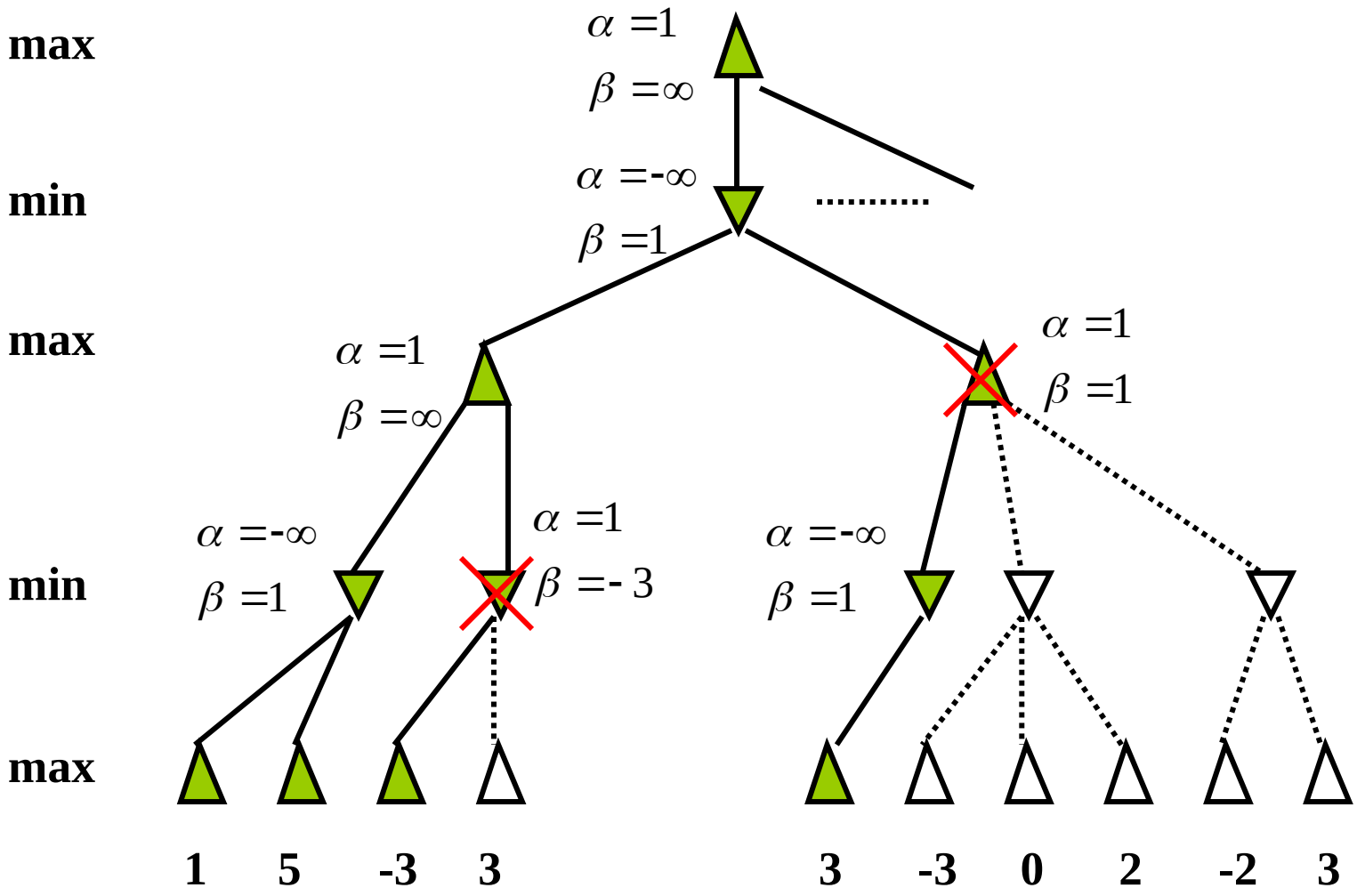
```
function minimax(node, depth, α, β, is_maximizing)

if node is a terminal node or depth = 0
    return the heuristic value of node

else if is_maximizing
    foreach child of node
        α := max(α, minimax(child, depth-1, α, β, false))
        if β ≤ α
            break
    return α

else
    foreach child of node
        β := min(β, minimax(child, depth-1, α, β, true))
        if β ≤ α
            break
    return β
```

# Alpha-beta pruning




Basado en Juris Viksna

# Alpha-beta pruning




Basado en Juris Viksna

# Entrada / Salida
## (para el laboratorio)

Manejo de archivos:

- **`open(+Path,+Mode,-Stream)`** ← Mode puede ser **`read`**, **`write`**, **`append`**, …
- **`close(+Stream)`**

- Luego se utilizan versiones de los predicados **`read`**, **`write`**, **`get`**, **`put`** que toman un stream como primer argumento.

# Entrada / Salida
## (para el laboratorio)

La forma fácil de utilizar E/S:

- Trabajar de a términos

**`read(+Stream,-Term)`** ← Lee un término entero del Stream (debe terminar en '.')

**`write(+Stream,+Term)`** ← Escribe un término entero en el Stream

No olvidar de escribir un punto al final:

**`write(Stream,'.')`**